Tools · Quick Reference Guide · Designing Tools the Model Can Actually Use

THE TOOL PROBLEM: People throw 50 vaguely-defined tools at the model and hope for the best.
Result: Loops · Failures · Hallucinations · Wasted tokens · Decision paralysis.

Core Principle: Tools are a form of communication. You are not just defining what a function does — you are teaching the model WHEN and HOW to use it. A tool without clear usage contracts forces the model to guess. Guessing produces drift.

SECTION 1 · The Three-Part Tool Definition Standard — Every tool must have all three parts

Part	What It Solves	What Goes Here	Bad Example	Good Example
1. TRIGGER LOGIC (When to Use This)	The 'guess the intent' problem. Without it: Model uses tool at wrong times, misses right times, activates on superficial keyword matches.	use_when: • Specific scenario 1 • Specific scenario 2 dont_use_when: • Use reasoning instead • Use different_tool instead • Edge case where this fails	✗ "name": "get_data" "description": "Gets data" Model has no idea when to use this vs. any other data-related tool.	✓ "name": "get_user_profile" use_when: • "Need specific person's info" • "Verify user before action" dont_use_when: • "Multiple users → search_users" • "Creating user → create_user"
2. NEGATIVE CONSTRAINTS (What NOT to Do)	Common failure modes and safety issues. Without it: Model makes predictable mistakes, exposes data, creates loops, ignores rate limits.	do_not: • Specific mistake to avoid • Input pattern that fails • Behavior causing loops safety: • Data exposure rules • Rate limits • Timeout thresholds	✗ No constraints defined. Model assumes: • Any input is valid • All fields safe to return • No limits apply Result: PII exposed, loops, crashes.	✓ do_not: • "Execute on null input" • "Assume column names" safety: • "Never return password fields" • "Limit to 100 rows max" • "Timeout after 5 seconds"
3. RETURN CONTRACT (What to Expect)	Unknown failure modes and recovery patterns. Without it: Model hallucinates error recovery, retries when it shouldn't, gives up when it should retry.	success: • Schema + example failure_modes: • error_type → message • recovery_action • retry: true/false • user_action (what to say)	✗ No failure modes defined. Model encounters error: → Doesn't know if retryable → Doesn't know what to tell user → Hallucinates recovery → Loop begins.	✓ rate_limit: error: "Rate limit exceeded" recovery: "Wait 60s, retry" retry: true user_action: "Sending in 60s" not_found: recovery: "Ask user to verify ID" retry: false

SECTION 2 · Tool Classification System — Every tool must be classified by risk before deployment

Class	Definition	Examples	Confirmation Rule	Class C Clarification
Class A Read-Only	No side effects. Safe to retry. Safe to use speculatively. Output does not change system state.	get_user search_documents calculate_statistics read_file query_database (SELECT only)	**No confirmation needed.** **Model may call freely.** **Safe to retry on failure.**	— (not applicable)
Class B State-Change	Irreversible or has side effects. Changes system state. Cannot be automatically undone.	delete_file send_email update_database process_payment post_to_api	**ALWAYS confirm with user before execution.** **Confirmation must include:** **1. What will change** **2. What cannot be undone** **3. Require specific acknowledgment** **✗ Weak: 'Are you sure?'** **✓ Strong: See Section 3**	— (not applicable)
Class C Computational	Tasks where accumulated steps, scale, or precision requirements make in-context reasoning unreliable. ⚠ NOT: 'tasks a model finds hard' NOT: complex multi-step reasoning NOT: sophisticated analysis Class C = error accumulates FASTER than reasoning can correct it.	360-month mortgage amortization Large dataset statistical analysis Cryptographic operations High-precision financial calculations Multi-variable optimization at scale	**Use dedicated computational tool.** **Do not attempt in-context reasoning.** **Verify output against known test cases.**	**CLASS C LITMUS TEST:** **'Would a small arithmetic error in step 3 silently corrupt all subsequent steps?'** **✓ YES → Class C (use tool)** **✗ NO → Probably a Skills or reasoning problem, not Class C** **Don't misclassify complex reasoning tasks as Class C. Complexity ≠ computational precision requirement.**

SECTION 3 · Class B Confirmation Dialog Pattern — 'Are you sure?' is not a safety mechanism

WHY WEAK CONFIRMATION FAILS: 'Are you sure?' trains users to click through without reading. It provides the appearance of safety without the substance. A strong confirmation forces the user to demonstrate they understand WHAT will change and WHAT cannot be undone — not just that they want to proceed.

✗ WEAK Confirmation (Do NOT use)

```
Examples of weak confirmation:

"Are you sure you want to delete this?"

"Confirm: Yes / No"

"This action cannot be undone. Continue?"

Why these fail:
• User doesn't know WHAT is being deleted
• User doesn't know WHAT specifically can't be undone
• No specific acknowledgment required
• Trains click-through behavior
• Provides legal cover, not actual safety
```

✓ STRONG Confirmation Template (Use this)

```
Strong confirmation must include:

1. WHAT WILL CHANGE (specific, not generic)
   "This will permanently delete user account #4521
    (jane.doe@company.com) and all 847 associated records."
2. WHAT CANNOT BE UNDONE
   "This cannot be recovered. There is no undo.
    Backups are retained for 30 days for admin recovery only."
3. SPECIFIC ACKNOWLEDGMENT REQUIRED
   "Type DELETE to confirm:" or
   "Type the account email to confirm:" or
   "Confirm the record count (847) to proceed:"
Why this works:
• Forces user to READ before confirming
• Specific acknowledgment proves comprehension
• Makes accidental confirmation nearly impossible
```

Tools · Quick Reference Guide · Designing Tools the Model Can Actually Use (Page 2)

SECTION 4 · Model-Driven vs Programmatic Tool Calling — For production agents, programmatic wins

Model-Driven Tool Calling	Programmatic Tool Calling ← Preferred for production
The agent reasons about what to do, decides which tool to call, generates that decision as output, and the runtime executes it. Problems: • Tool selection depends on model belief state • Deliberation tokens enter the evidential stream • Error handling lives in context (fragile) • Drift affects tool choice over long sessions • Works well for simple tasks and exploration	Your code decides which tool to call based on explicit logic. The model provides intelligence, code provides control. Advantages: • Tool selection guaranteed by logic, not belief state • Deliberation tokens stay OUT of evidential stream • Error handling lives in code (reliable) • Enforces Class A/B/C classification architecturally • Required for production multi-agent systems

⚠ THE EVIDENTIAL STREAM PROBLEM — Why deliberation tokens in context cause drift

When a model reasons about which tool to call, that reasoning becomes tokens in the context window. Those tokens are now evidence — and they accumulate.

Example: Agent is deciding between delete_record and archive_record. Model reasons: 'The user probably wants a clean system, deleting is cleaner than archiving...' That reasoning now sits in context as evidence toward a 'prefer deletion' prior. In a long session or multi-agent system, this accumulation pushes behavior in a direction the original spec never intended — without any single decision being obviously wrong.

Programmatic calling removes this: code decides the tool, model never deliberates about tool selection, no deliberation tokens enter the stream. The evidential context stays clean.

FOUR PROGRAMMATIC CALLING PATTERNS

Pattern	When to Use	How It Works	Example Use Case	Key Rule

Sequential Calling	Steps have dependencies.
	Step N requires output of Step N-1.
		`Code enforces order.`
		`Each tool result gates the next call.`
		`No skipping steps.`
			`get_customer → verify_identity → process_refund`
			
			`Identity verification MUST complete before refund is possible.`
				Never let the model decide step order when order has business or safety implications.

Conditional Calling	Which tool to call depends on a condition that code can evaluate.	`IF/ELSE in code determines tool.`
		`Model provides the value being tested.`
		`Code makes the routing`
			`IF order_value > $500:`
			`  → escalate_to_human`
			`ELSE:`
			`  → process_automatically`
				Conditions with business rule implications belong in code, not in model reasoning.

Parallel Calling	Multiple independent tools needed.
	No dependency between them.
		`Emit multiple tool calls simultaneously.`
		`Aggregate results before synthesis.`
			`Simultaneously:`
			`get_inventory + get_pricing + get_supplier`
			`Then synthesize all three results.`
				Only parallelize truly independent calls. Hidden dependencies cause race conditions.

Error Handling & Fallback	Tool may fail.
	Recovery path must be reliable.
		`TRY primary tool.`
		`CATCH specific error types.`
		`FALLBACK to alternative or escalate.`
			`TRY: search_live_database`
			`CATCH timeout: search_cache`
			`CATCH cache_miss: notify_user_and_log`
				Error recovery in code is deterministic.
				Error recovery in prompt is probabilistic.

SECTION 5 · The Five Tool Antipatterns — These cause most production failures

Antipattern	What It Looks Like	Why It Fails	→	Fix
1. The '50 Tools Dump'	`Every available function exposed as a tool regardless of relevance.` `Tools with overlapping descriptions.`	Decision paralysis — model spends context deliberating instead of acting. Vague descriptions cause misrouting. Deliberation tokens contaminate evidential stream. Performance degrades with tool count.	→	Maximum 10-15 tools per agent. Each agent gets only tools for ITS role. Split into specialized agents if more tools needed. Write descriptions that clearly differentiate similar tools.

2. The 'Vague Description'	`"get_data" · "process_item" · "handle_request"`
	
	`Descriptions that could apply to 3+ tools.`
		Model cannot differentiate tools.
		Selects based on superficial keyword matching.
		Wrong tool called for right task.
		Produces subtly wrong results that look correct.
			→	Name must include: what + which system.
				Description must include: when to use vs. similar tools.
				Add dont_use_when pointing to alternatives.
				Test: give model ambiguous request — does it pick correctly?

3. The 'Hidden Risk'	`Class B tools (state-change) described as if Class A.` `No warning that action is irreversible.`	Model uses destructive tool speculatively. No confirmation triggered. Data lost, emails sent, payments processed. User never warned.	→	Every Class B tool must declare: • 'This action is irreversible' • What state will change • Confirmation requirement Class in system prompt: 'Tools marked [B] require user confirmation.'
4. The 'Error Black Hole'	`No failure modes defined in Return Contract.` `Generic error message: 'Operation failed'`	Model doesn't know if error is retryable. Model doesn't know what to tell user. Model hallucinates recovery steps. Retry loops begin. Context fills with failed attempts.	→	Every tool needs failure modes: • transient (retry) vs. permanent (don't retry) • isRetryable: true/false • recovery_action for each error type • user_action: what to tell the user
5. The 'Everything's a Tool'	`Using tools for tasks that are faster and more reliable with in-context reasoning.` `Tools called for simple calculations, formatting, or knowledge questions.`	Unnecessary latency from tool calls. Token overhead from tool definitions. Context filled with tool results for simple tasks. Over-engineered solution for simple problem.	→	Decision framework: • Can reasoning solve this reliably? → Use reasoning • Does this require live data? → Tool • Does this require computation precision? → Class C tool • Does this change system state? → Class B tool with confirmation

✦ REASONING vs TOOLS DECISION FRAMEWORK — Use the minimum tool necessary

Ask these questions in order. Stop at the first YES.

1. Can in-context reasoning solve this reliably and accurately? → YES: Use reasoning. No tool needed.

2. Does this require live/external data the model doesn't have? → YES: Use Class A read tool.

3. Does this require computational precision where small errors cascade? → YES: Use Class C computational tool.
   (Remember: complexity ≠ Class C. Only use Class C when accumulated error is the specific risk.)

4. Does this change system state or have irreversible consequences? → YES: Use Class B tool WITH strong confirmation dialog.

5. Is this task genuinely beyond what tools 1-4 cover? → Reconsider scope. You may have a Skills or Spec problem, not a tool problem.